

Data-Driven Control for Cloud Resources

MAO, Yu

118010224

The Report for CSC6032 Advanced Operating Systems

The Chinese University of Hong Kong, Shenzhen

May 2026

Abstract

Modern cloud platforms host many long-running applications (LRAs), including online services, databases, stream-processing pipelines, and machine-learning workloads. These applications are usually business-critical and constrained by service-level objectives (SLOs). Cloud resource management is therefore not only a placement problem, but a continuous control problem: the system must decide where workloads run, when they should move, and how much capacity should be provisioned under scale, dynamics, and uncertainty.

This thesis develops data-driven cloud resource control through three representative systems: METIS, TETRIS, and OPTSCALER [15, 17, 18]. The three systems are organized as a progressive trajectory. METIS learns admission-time placement policies for LRA containers, replacing manually specified placement constraints with reinforcement learning and using hierarchical reinforcement learning to reduce the large combinatorial action space. TETRIS moves from one-time placement to runtime rescheduling. It models proactive, migration-aware scheduling as a finite-horizon model predictive control (MPC) problem that jointly considers future load balance and migration cost. OPTSCALER further extends the control scope to cluster-level autoscaling. It combines proactive workload prediction, reactive CPU estimation, robust MPC, and chance constraints to handle prediction error, metric noise, and co-location uncertainty.

The thesis makes three main contributions. First, it develops the three systems under a common feedback-control abstraction. Second, it presents their core methods: reinforcement-learning-based placement, MPC-based rescheduling, and robust MPC-based autoscaling. Third, it synthesizes them into a unified design view in which cloud resource management evolves from data-driven placement, to prediction-driven runtime control, and finally to feedback- and uncertainty-aware robust orchestration. This synthesis suggests that future cloud control planes should coordinate placement, migration, and scaling decisions within one Kubernetes-native framework while improving theoretical guarantees, online overhead, and multi-resource awareness.

Contents

Abstract	i
List of Figures	v
List of Tables	vii
1 Introduction	1
1.1 Cloud Resource Management in Modern Clouds	1
1.2 Three Challenges of Cloud Resource Orchestration	2
1.3 Thesis Contributions	3
1.4 Thesis Organization	4
2 Background and Related Work	5
2.1 Long-Running Applications and SLO-driven Management	5
2.2 Feedback Control as a Common Abstraction	6
2.3 Scheduling, Rescheduling, and Autoscaling as Unified Control Problems . .	6
2.3.1 Initial Placement and Learning-based Scheduling	7
2.3.2 Migration-aware Rescheduling	7
2.3.3 Autoscaling under Uncertainty	8
3 Learning-based Initial Placement: Metis	10
3.1 Motivation	10
3.2 Problem Formulation: Scheduling as Reinforcement Learning	11
3.3 Cluster Environment Simulator	12
3.4 Hierarchical Reinforcement Learning for Scalability	13
3.4.1 Dedicated On-the-spot Training	13
3.4.2 Experience Replay and Reward Design	14
3.5 Overall System Architecture	14
3.6 Evaluation and Results	14
3.6.1 Experimental Setup	14
3.6.2 Throughput and Placement Quality	14
3.6.3 Scalability of Hierarchical Learning	15
3.7 Summary and Limitations	16
4 Proactive Rescheduling and Migration: Tetris	17

4.1	Motivation	17
4.2	Discrete-time Dynamic Model	17
4.3	Container Resource Prediction	19
4.4	MPC Formulation for Migration-aware Scheduling	19
4.5	Efficient Solving through Candidate Filtering and Monte Carlo Sampling . .	20
4.6	Evaluation and Results	21
4.6.1	Experimental Setup	21
4.6.2	Load Balancing and Migration Cost	21
4.6.3	Invalid Migration Reduction	22
4.6.4	SLO Violations and Runtime Overhead	23
4.7	Summary and Limitations	23
5	Robust Collaborative Autoscaling: OptScaler	24
5.1	Motivation: From Rescheduling to Autoscaling under Uncertainty	24
5.2	Cloud Deployment and Workload Characteristics	24
5.3	Framework Overview	25
5.4	Proactive Workload Prediction	25
5.4.1	Long-term Periodic Block	26
5.4.2	Short-term Local Block	26
5.4.3	Comparison with Tetris Prediction	27
5.5	Reactive CPU Estimator and Feedback Adaptation	27
5.6	Robust MPC with Chance Constraints	28
5.7	Evaluation and Results	28
5.7.1	Prediction Accuracy	28
5.7.2	Autoscaling Framework Comparison	29
5.7.3	Sensitivity and Robustness	31
5.7.4	Online Deployment at Alipay	32
5.8	Summary and Limitations	32
6	Synthesis and Comparative Discussion	34
6.1	Evolution of Control Scope: Placement to Migration to Scaling	34
6.2	Evolution of Methodology: RL to MPC to Robust MPC	34
6.3	Evolution of SLO Guarantees	34
6.4	Data, Prediction, and Feedback	35
6.5	System-level Trade-offs	35
6.5.1	Optimality versus Scalability	35
6.5.2	Responsiveness versus Stability	35
6.5.3	Resource Cost versus SLO Safety	35
6.5.4	Interpretability versus Black-box Learning	36
6.6	Toward an Integrated Cloud Resource Controller	36
6.7	Limitations of This Synthesis	36

7	Conclusion and Future Work	37
7.1	Conclusion	37
7.2	Future Work	37
7.2.1	Multi-resource Robust Control	37
7.2.2	Low-overhead Online Learning	38
7.2.3	Stronger Theoretical Guarantees	38
	Bibliography	39

List of Figures

1.1	Overview of cloud resource architecture, showing the hierarchy among containers, pods, nodes, and clusters.	2
1.2	Problem–solution overview for cloud resource orchestration: scale, dynamics, and uncertainty motivate METIS, TETRIS, and OPTSCALER, forming a progression from learned placement to predictive migration and robust autoscaling.	3
2.1	Unified LRA lifecycle and feedback-control loop: workload arrival, initial placement, execution, rescheduling/migration, autoscaling, monitoring, prediction/estimation, optimization, and actuation, annotated with METIS, TETRIS, and OPTSCALER.	6
2.2	Illustration of invalid migration, where containers are moved back and forth among servers within a short time window.	8
3.1	LRA scheduling workflow: incoming container launching requests are buffered, batched, and placed onto cluster nodes.	10
3.2	Performance interactions among LRAs, showing that affinity and anti-affinity relationships can have different magnitudes and may conflict.	11
3.3	Simulator-assisted training and deployment loop in METIS.	12
3.4	Hierarchical reinforcement-learning architecture for scalable container placement.	13
3.5	Performance comparison of METIS, Medea, and Paragon in an EC2 cluster.	15
3.6	Scalability of hierarchical learning in METIS.	16
4.1	Illustrative example comparing MPC-based TETRIS and RL-based METIS+.	18
4.2	Examples of periodic and aperiodic container resource-consumption patterns.	19
4.3	Overall architecture of TETRIS, including resource prediction, MPC-based scheduler, and container migration module.	20
4.4	Performance comparison of TETRIS, Sandpiper, and Metis+ in terms of cumulative load imbalance and migration cost.	22
4.5	Cumulative number of invalid migrations under TETRIS, Sandpiper, and Metis+.	23
5.1	Periodic workload patterns and the linear relationship between unit workload and CPU utilization.	25

5.2	Overall architecture of OPTSCALER: proactive module, reactive module, optimization module, cloud system, and feedback.	26
5.3	Workload prediction model in the proactive module, including long-term periodic block and short-term local block.	27
5.4	Workload prediction performance of OPTSCALER compared with baseline predictors.	29
5.5	CPU utilization comparison across autoscaling frameworks.	30
5.6	Sensitivity of feedback strength η , showing the trade-off between resource cost and SLO violation rate.	31
5.7	Online autoscaling result at Alipay, showing installed nodes and CPU utilization over time.	32

List of Tables

5.1	Workload prediction accuracy in OPTSCALER. Lower WAPE is better.	29
5.2	Autoscaling comparison in OPTSCALER. Lower values are better.	30
5.3	Ablation comparison between OPTSCALER and HAS with NBEATS. Lower values are better.	31

Chapter 1

Introduction

1.1 Cloud Resource Management in Modern Clouds

Cloud computing has become the dominant infrastructure for deploying large-scale online services. Instead of provisioning a fixed number of physical servers for a single application, modern cloud platforms organize computing resources into elastic clusters and allocate them dynamically to many co-located workloads. This abstraction allows application owners to scale services, tolerate faults, and reduce operational cost, while cloud providers can improve utilization through statistical multiplexing. However, the same abstraction also introduces a difficult operating-system-level problem: the infrastructure must decide where workloads should run, when they should be moved, and how many resources should be provisioned, all while respecting performance objectives.

At the lowest deployment level, a cloud application is usually packaged into containers. A container provides an isolated runtime environment with application code, dependencies, and configuration. In Kubernetes-like systems, one or more containers may be grouped into a pod, and pods are scheduled onto nodes, where a node is a physical or virtual machine [8]. A cluster consists of many such nodes coordinated by a control plane. The hierarchy in Figure 1.1 fixes this terminology before the thesis moves from individual workloads to cluster-level control. Although these concepts are now standard, their interactions create complex resource-management decisions. A container placement decision affects CPU contention, memory pressure, cache interference, network locality, and failure isolation. A migration decision affects both the source and destination machines, while also imposing service disruption and state-transfer overhead. A scaling decision changes the number of active nodes or replicas, influencing both SLO satisfaction and resource cost.

The workloads considered in this thesis are long-running applications (LRAs). Unlike batch jobs that terminate after a finite computation, LRAs remain active for long periods, often serving external user requests. Examples include web services, storage systems, databases, stream-processing services, machine-learning inference servers, recommendation systems, and payment services. LRAs are first-class workloads in production clusters because they are typically user-facing and business-critical. Their performance is measured by service-level objectives (SLOs), such as average latency, tail latency, throughput, or CPU-utilization thresholds that indirectly protect latency.

The objective of cloud resource management is therefore multi-dimensional. A sched-

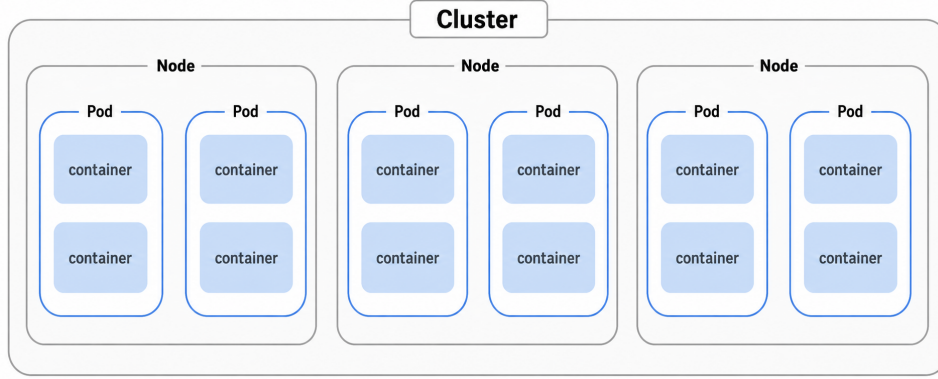


Figure 1.1: Overview of cloud resource architecture, showing the hierarchy among containers, pods, nodes, and clusters.

uler should place containers so that throughput is high and interference is low. A resched-
 uler should react to workload changes without causing excessive migration cost. An
 autoscaler should provision enough capacity to absorb bursts but avoid wasteful over-
 provisioning. These objectives are tightly coupled. For example, a desired placement may
 reduce the need for migration; a desired migration policy may delay autoscaling; and a ro-
 bust autoscaler may reduce the chance that local placement errors turn into SLO violations.
 This thesis therefore treats scheduling, rescheduling, and autoscaling as connected stages
 of a single cloud-resource control pipeline.

1.2 Three Challenges of Cloud Resource Orchestration

This thesis organizes cloud resource orchestration around three fundamental challenges:
 scale, dynamics, and uncertainty. These challenges motivate the three systems developed
 in the thesis and provide its logical structure.

The first challenge is *scale*. A large production cluster may contain hundreds or thou-
 sands of machines and tens of thousands of containers. Even a simplified container-
 placement problem is combinatorial: if T incoming containers can be placed on N nodes,
 the naive action space scales as N^T . Realistic placement decisions are harder because con-
 tainer interactions are not additive. Two applications may benefit from co-location because
 of data locality, or suffer from co-location because of resource interference. Traditional
 schedulers often encode such preferences as affinity and anti-affinity constraints, but these
 constraints are qualitative and manually specified. A scheduler that minimizes the number
 of violated constraints may not maximize actual throughput. METIS addresses this scale
 problem by learning performance-driven placement policies from data and by decompos-
 ing large placement decisions into a hierarchy of smaller choices.

The second challenge is *dynamics*. LRAs are long-lived, but their resource consumption
 changes over time as user demand fluctuates. A placement that is optimal at admission
 time may become poor after several hours. Reactive load balancers can migrate containers
 away from overloaded nodes, but if they only optimize the current time slot, they may

move the same containers back and forth, creating invalid migrations. Such migrations waste resources and may violate SLOs, particularly for stateful services. TETRIS addresses this problem by formulating rescheduling as finite-horizon MPC. Instead of reacting only to the current overload, it predicts future resource consumption and jointly considers load balancing and migration cost.

The third challenge is *uncertainty*. Prediction is useful but imperfect. Workload forecasts can deviate from actual demand, CPU-utilization measurements can be noisy, and co-located applications can interfere in ways that change over time. A purely proactive autoscaler may under-provision when predictions are too low, while a purely reactive autoscaler may respond too late. Hybrid autoscalers that independently combine proactive and reactive modules can suffer from conflicting decisions. OPTSCALER addresses this issue through collaborative autoscaling. Its proactive module predicts future workload, its reactive module continuously estimates the relationship between workload and CPU utilization, and its optimization module uses MPC with chance constraints to produce robust scaling plans. The resulting problem–solution map is summarized in Figure 1.2: scale leads to learned placement, dynamics leads to predictive migration, and uncertainty leads to robust autoscaling.

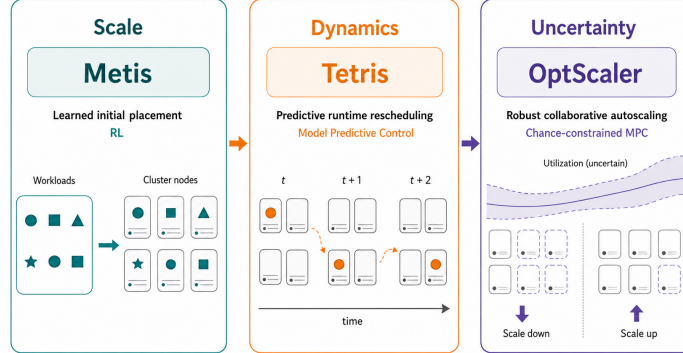


Figure 1.2: Problem–solution overview for cloud resource orchestration: scale, dynamics, and uncertainty motivate METIS, TETRIS, and OPTSCALER, forming a progression from learned placement to predictive migration and robust autoscaling.

1.3 Thesis Contributions

This thesis develops a coherent approach to data-driven cloud resource control through three connected systems. The contributions are organized as follows.

First, the thesis introduces learning-based initial placement. METIS observes that placement constraints are an indirect and incomplete representation of performance. It therefore learns scheduling policies that map cluster states to placement actions and uses observed or simulated performance as the reward. The technical difficulty is scalability. Direct RL over all nodes is infeasible, so METIS partitions the cluster recursively and makes a sequence of smaller placement decisions. This contribution addresses the admission-time question:

where should newly arriving containers be placed?

Second, the thesis develops predictive runtime rescheduling. TETRIS starts from the limitation of one-time placement. Since LRAs have time-varying resource consumption, the system must periodically reconsider the mapping from containers to servers. However, migration is not free. TETRIS therefore builds a discrete-time dynamic model, predicts container resource demand, and optimizes a cost function containing both load imbalance and migration cost. This contribution addresses the runtime question: *which containers should be migrated, and where should they go?*

Third, the thesis presents robust collaborative autoscaling. OPTSCALER moves the control object from individual containers to cluster capacity. It recognizes that both reactive and proactive autoscaling are incomplete. Reactive methods are delayed; proactive methods are vulnerable to prediction error; independent hybrid methods may conflict. OPTSCALER addresses these limitations by letting proactive and reactive modules provide dynamic inputs to a single optimization module. This contribution addresses the capacity-planning question: *how many resources should be provisioned under uncertainty?*

Fourth, the thesis synthesizes the three systems into an integrated trajectory. The control scope evolves from placement to migration to scaling. The methodology evolves from RL to MPC to robust MPC. The SLO treatment evolves from soft reward to hard deterministic constraints to probabilistic chance constraints. This synthesis is the conceptual backbone of the thesis.

1.4 Thesis Organization

The rest of the thesis is organized as follows. Chapter 2 introduces long-running applications, SLO-driven management, a feedback-control abstraction, and related work in placement, rescheduling, and autoscaling. Chapter 3 presents METIS, focusing on RL-based and HRL-based initial placement. Chapter 4 presents TETRIS, focusing on MPC-based proactive rescheduling with migration cost. Chapter 5 presents OPTSCALER, focusing on collaborative autoscaling under uncertainty. Chapter 6 synthesizes the three systems through control scope, methodology, SLO guarantees, data usage, and trade-offs. Chapter 7 concludes the thesis and discusses future research directions.

Chapter 2

Background and Related Work

2.1 Long-Running Applications and SLO-driven Management

Modern cloud platforms run applications on shared clusters composed of containers, pods, and nodes. A container packages an application instance and its dependencies; a pod is the Kubernetes-style deployment unit that may contain one or more containers; and a node is the physical or virtual machine that provides CPU, memory, storage, and network resources [8]. Production cluster managers such as Borg expose similar resource-allocation problems even when their implementation details differ [14]. In all cases, the control plane must translate application-level demand into machine-level placement, migration, and scaling decisions.

This thesis focuses on long-running applications (LRAs). Unlike batch jobs that terminate after a finite computation, LRAs persist for long periods and often serve external requests. Examples include web services, databases, stream-processing services, recommendation systems, and machine-learning inference services. LRAs are difficult to manage because their demand changes over time, their performance is externally visible, and their service-level objectives (SLOs) must be protected while sharing infrastructure with other workloads.

The lifecycle of an LRA naturally contains several control stages. When a workload arrives, the system must decide its initial placement. During execution, changing resource consumption may require migration or rescheduling. At a larger time scale, aggregate demand may require adding or removing nodes. Monitoring feedback then updates future decisions. Figure 2.1 organizes these stages as a feedback-control loop and shows why METIS, TETRIS, and OPTSCALER fit naturally into one lifecycle: initial placement, runtime migration, and cluster-level autoscaling.

SLO-driven management is the reason these stages are coupled. A purely cost-driven controller may over-consolidate workloads and trigger latency spikes. A purely safety-driven controller may over-provision resources and waste capacity. Production systems must operate between these extremes. METIS uses performance rewards to learn placements that improve application-level metrics. TETRIS protects SLOs through CPU and memory capacity constraints during migration planning. OPTSCALER regulates CPU utilization around a desired threshold while limiting the probability of SLO violation under uncertainty.

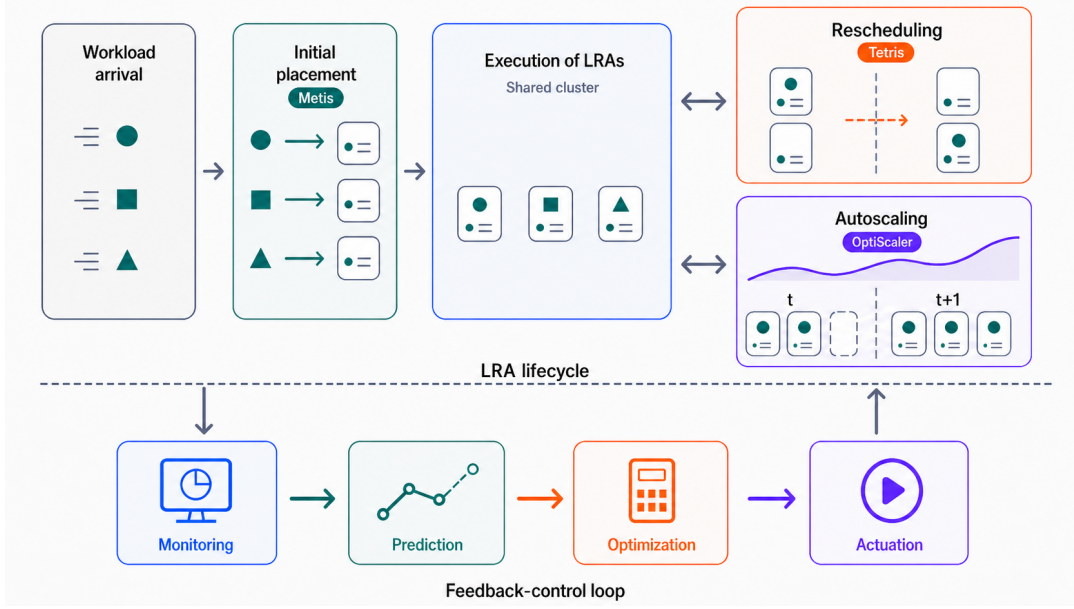


Figure 2.1: Unified LRA lifecycle and feedback-control loop: workload arrival, initial placement, execution, rescheduling/migration, autoscaling, monitoring, prediction/estimation, optimization, and actuation, annotated with METIS, TETRIS, and OPTSCALER.

2.2 Feedback Control as a Common Abstraction

A useful abstraction for cloud resource management is the feedback-control loop. The cloud environment produces observations, such as current placement, node utilization, application demand, and SLO status. A prediction or estimation module transforms these observations into forecasts: expected throughput, future resource consumption, future workload, or CPU-utilization response. A controller or optimizer then selects actions, such as placing containers, migrating containers, adding nodes, or removing nodes. After actuation, the environment changes and produces new feedback.

Under this abstraction, METIS is a learning-based placement controller. It observes the current cluster state and a batch of incoming containers, chooses placement actions, and receives a delayed reward after the batch has been placed. TETRIS is a predictive migration controller. It observes the current placement, predicts future resource consumption, and solves a finite-horizon optimization problem to choose the next migration action. OPTSCALER is a robust scaling controller. It observes historical workload and real-time CPU feedback, estimates future utilization under different node counts, and solves a chance-constrained MPC problem to determine scaling plans.

2.3 Scheduling, Rescheduling, and Autoscaling as Unified Control Problems

The three systems differ in control object and time scale, but they share the same operating-system question: how should a control plane allocate shared resources under imperfect information? Initial placement decides where newly arriving containers should run. Runtime rescheduling decides whether already running containers should migrate. Autoscaling de-

cides how much cluster capacity should be provisioned. These decisions are interdependent because placement affects future interference, migration affects node utilization, and scaling changes the feasible placement space.

2.3.1 Initial Placement and Learning-based Scheduling

Traditional placement systems rely on heuristics or operator-specified constraints. Affinity constraints express that two containers should be co-located, often for locality. Anti-affinity constraints express that two containers should be separated, often to reduce interference or improve availability. These constraints are interpretable and compatible with production policies, but they are only proxies for true performance. They do not quantify whether satisfying a constraint improves throughput slightly or dramatically, and conflicting constraints may lead to placements that satisfy many rules but still perform poorly.

Learning-based scheduling replaces such intermediate objectives with direct performance optimization. Reinforcement learning (RL) is attractive because scheduling can be formulated as sequential decision making: the scheduler observes a state, takes an action, receives a reward, and updates its policy [12]. More formally, an RL problem is often represented as a Markov decision process with state space $\mathcal{S}$, action space $\mathcal{A}$, transition rule P , reward function r , and discount factor γ . A policy $\pi(a | s)$ chooses an action for each state, and training seeks a policy that maximizes expected cumulative reward. In cluster scheduling, the state contains the current placement and workload mix, the action is a placement decision, and the reward can be throughput or SLO satisfaction.

This abstraction fits METIS, but the difficulty is scale. A production cluster has many nodes, many application types, and many possible co-location patterns. A flat policy over all nodes gives a large action space, while a full state vector over all nodes gives a large input space. Hierarchical RL addresses this by decomposing a long or large decision into smaller decision levels; the option framework formalizes such temporal abstraction in RL [13]. METIS uses the same design intuition spatially: it combines simulator-assisted training with hierarchical reinforcement learning, so that a large placement decision is decomposed into smaller learned choices.

2.3.2 Migration-aware Rescheduling

Initial placement is only the first control decision. LRAs are time-varying, so a cluster can become imbalanced even if the initial placement was appropriate. Migration-aware rescheduling restores balance by moving containers, but migration is not free: it may require service pause, state transfer, reconstruction, and warm-up. For stateful services, even short disruptions can affect SLOs.

A harmful migration pattern is invalid migration, where a container is moved away from a server and then moved back within a short interval, as illustrated in Figure 2.2. This usually means the scheduler reacts to transient load without understanding future demand. Earlier systems such as Sandpiper demonstrate the value of migration for resource

management, but this back-and-forth behavior also makes clear why reactive migration alone is insufficient for long-running cloud services [16].

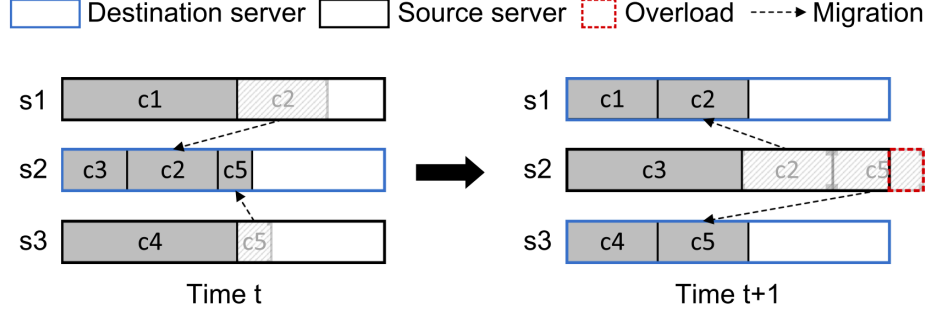


Figure 2.2: Illustration of invalid migration, where containers are moved back and forth among servers within a short time window.

Model predictive control (MPC) provides a more suitable control structure [3, 10]. At time t , an MPC controller observes the current state x_t , predicts future states over a finite horizon H , solves an optimization problem for an action sequence $(u_t, \dots, u_{t+H-1})$, executes only the first action u_t , and repeats after new feedback arrives. This receding-horizon structure is useful for operating systems because it combines prediction with correction: the controller can reason about near-future load without committing to a long open-loop plan. TETRIS applies this idea to migration-aware container scheduling, where the objective combines load imbalance and migration cost, and the constraints encode CPU and memory capacities.

Prediction is therefore part of the control problem rather than a separate convenience. TETRIS uses ARIMA for periodic resource series and LSTM for aperiodic ones, relying on classical time-series forecasting and neural sequence modeling [1, 7]. The finite MPC horizon limits the damage of prediction error: if a forecast is wrong, the next control step observes the new cluster state and replans.

2.3.3 Autoscaling under Uncertainty

Autoscaling adjusts provisioned capacity according to demand. Reactive autoscalers use current metrics, such as CPU utilization, and scale after thresholds are crossed. They are simple and robust to modeling errors, but they can respond too late. Proactive autoscalers predict future workload and scale in advance. They can avoid delayed response, but prediction error can cause under-provisioning or over-provisioning. Hybrid autoscalers combine both ideas, but independent proactive and reactive modules can still produce inconsistent decisions. Workload autoscaling systems such as Autopilot show the importance of automated capacity management in production clusters [11], while OPTSCALER focuses on robust collaborative autoscaling for co-located LRAs.

OPTSCALER treats autoscaling as an uncertainty-aware control problem. Deterministic MPC optimizes for one predicted trajectory, but autoscaling decisions are exposed to fore-

cast error, measurement noise, and co-location interference. Robust MPC extends MPC by optimizing under uncertainty or by tightening constraints so that decisions remain feasible when disturbances occur [9]. OPTSCALER follows this direction at the capacity level: the proactive module forecasts future workload, the reactive module estimates how workload maps to CPU utilization using real-time feedback, and the optimization module combines these inputs in one robust MPC problem.

The key safety primitive is a chance constraint, a probabilistic constraint introduced in stochastic programming by Charnes and Cooper [4]. Instead of requiring a deterministic inequality such as $c_{t+h} \leq c^*$ under a single forecast, a chance constraint requires

$$\Pr(c_{t+h} \leq c^*) \geq 1 - \rho, \quad (2.1)$$

where ρ is the tolerated violation probability. This is a natural fit for SLO-driven autoscaling: the controller does not claim that overload is impossible, but it explicitly controls the acceptable risk level. This design makes the scaling action traceable to forecasts, feedback estimates, objectives, and probabilistic safety constraints.

Chapter 3

Learning-based Initial Placement: Metis

3.1 Motivation

The first technical problem is initial placement. When new LRA containers arrive, the scheduler must decide where to place them. This decision is difficult because LRAs interact in complex ways. Co-locating two containers can improve performance if they communicate frequently, but it can also degrade performance if they contend for CPU cache, memory bandwidth, disk I/O, or network resources. Traditional schedulers encode these relationships as placement constraints. Affinity encourages co-location; anti-affinity discourages it. However, constraints are only qualitative. They do not say whether violating one affinity constraint costs 1% throughput or 50% throughput. Consequently, a scheduler that satisfies the largest number of constraints may still produce a poor placement.

METIS begins from this observation and proposes a direct learning approach. Instead of asking operators to specify qualitative constraints, it learns from workload logs or profiling data how placements affect performance. It then trains an RL scheduler that maps cluster states to placement actions. The reward can be throughput, SLO satisfaction, utilization, or a combination of metrics. This formulation is appealing because it optimizes the metric that matters, rather than an intermediate hand-written proxy. In the LRA launch workflow in Figure 3.1, the learned scheduler is inserted exactly at the point where batched container requests must be mapped onto cluster nodes.

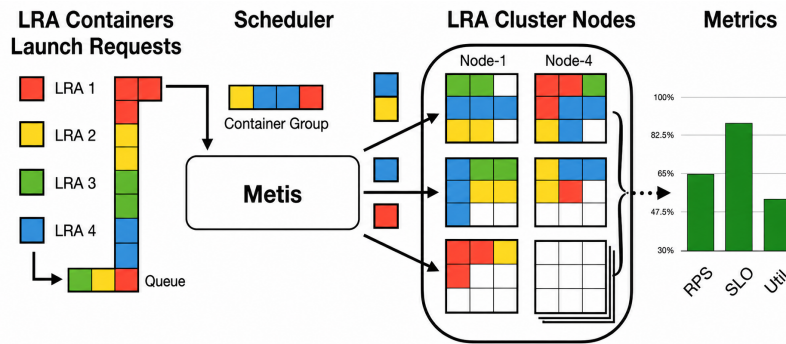


Figure 3.1: LRA scheduling workflow: incoming container launching requests are buffered, batched, and placed onto cluster nodes.

The main technical obstacle is scale. If a scheduler directly chooses one of N machines for each of T containers, the number of possible placements is enormous. Furthermore,

the state representation must capture the number and type of containers already running on each node. A naive RL policy would be too expensive to train and too slow to use. The interaction pattern in Figure 3.2 shows why a simpler constraint-counting approach is also insufficient: affinity and anti-affinity relationships have different magnitudes and may conflict. METIS therefore introduces a hierarchical approach that decomposes the placement decision into smaller sub-decisions while still learning quantitative performance effects.

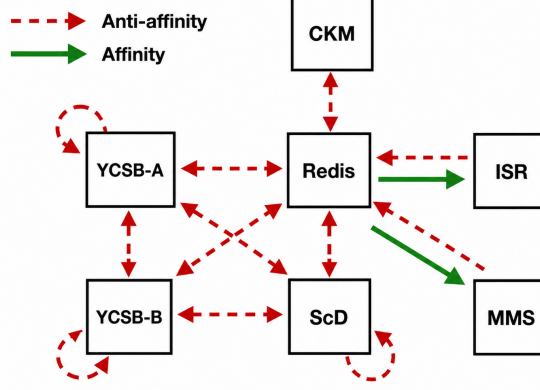


Figure 3.2: Performance interactions among LRAs, showing that affinity and anti-affinity relationships can have different magnitudes and may conflict.

3.2 Problem Formulation: Scheduling as Reinforcement Learning

METIS models LRA placement as an episodic RL problem. A scheduling episode corresponds to placing a group of T incoming containers. At step t , the scheduler places one container c_t onto a node. After all containers in the group are placed, the scheduler receives a reward based on the quality of the full placement.

Let the cluster contain N nodes and M applications. The current container c_t is encoded as a one-hot vector

$$e_t = (e_{t,1}, e_{t,2}, \dots, e_{t,M}), \quad (3.1)$$

where $e_{t,j} = 1$ if c_t belongs to application j . The state of node n is represented as

$$v_n = (v_{n,1}, v_{n,2}, \dots, v_{n,M}), \quad (3.2)$$

where $v_{n,j}$ is the number of containers of application j currently running on node n . The global state is the concatenation

$$s_t = \langle e_t, v_1, v_2, \dots, v_N \rangle. \quad (3.3)$$

The action is the selected node

$$a_t \in \{1, 2, \dots, N\}. \quad (3.4)$$

The policy network $\pi_\theta(a_t | s_t)$ outputs a probability distribution over nodes. Since the quality of a placement depends on the whole batch, intermediate rewards are often zero

and the final reward R is computed after the group is scheduled. A policy-gradient update can be written at a high level as

$$\theta \leftarrow \theta + \alpha \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (R - b), \quad (3.5)$$

where α is the learning rate and b is a baseline used to reduce variance.

3.3 Cluster Environment Simulator

Training an RL scheduler directly in a real cluster is impractical. Each training episode would require launching containers, waiting for them to stabilize, measuring performance, and then repeating the process thousands of times. METIS addresses this problem by building a cluster environment simulator. The simulator predicts performance metrics, such as request throughput or latency, from the co-location vector of containers on each machine.

The simulator is deliberately built at the performance-metric level rather than the microarchitectural level. Modeling every source of interference, including cache contention, network contention, disk I/O, and memory bandwidth, would be too complicated and often impossible from production traces. Instead, METIS logs observed placements and performance outcomes, then trains a regression model to approximate the mapping from placement to performance. METIS uses Random Forests because they can provide accurate predictions with relatively limited training data and are robust to repeated samples [2].

This simulator plays two roles. During training, it acts as the RL environment, allowing the policy to explore many placements without deploying them. During operation, profiler feedback from the real cluster can refine the simulator. Thus, METIS is not a purely offline learning system; the simulator-assisted loop in Figure 3.3 keeps a feedback path from real performance to the model used for future training.

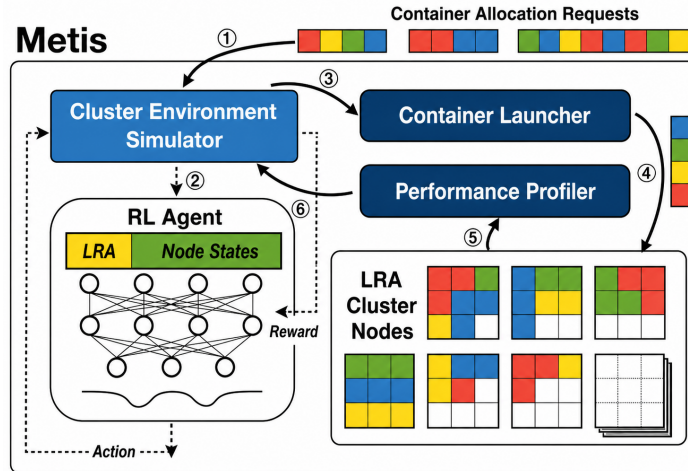


Figure 3.3: Simulator-assisted training and deployment loop in METIS.

3.4 Hierarchical Reinforcement Learning for Scalability

The key design in METIS is hierarchical reinforcement learning. The scheduler does not directly select a machine from the full cluster. Instead, it recursively partitions the cluster. At the top level, the policy chooses one sub-cluster among K partitions. At the next level, it chooses one sub-partition inside the selected sub-cluster. This process continues until a single machine is selected. Therefore, one large N -way decision is replaced by a sequence of smaller K -way decisions.

This design has three benefits. First, it reduces the action space at each step. Second, it allows subproblems at the same level to share structure, so a learned sub-scheduler can be reused across similar partitions. Third, it enables pretrained sub-schedulers for small sub-clusters, reducing online training time. Intuitively, HRL converts a flat placement problem into the tree-structured decision process shown in Figure 3.4, which is what makes large-cluster placement tractable.

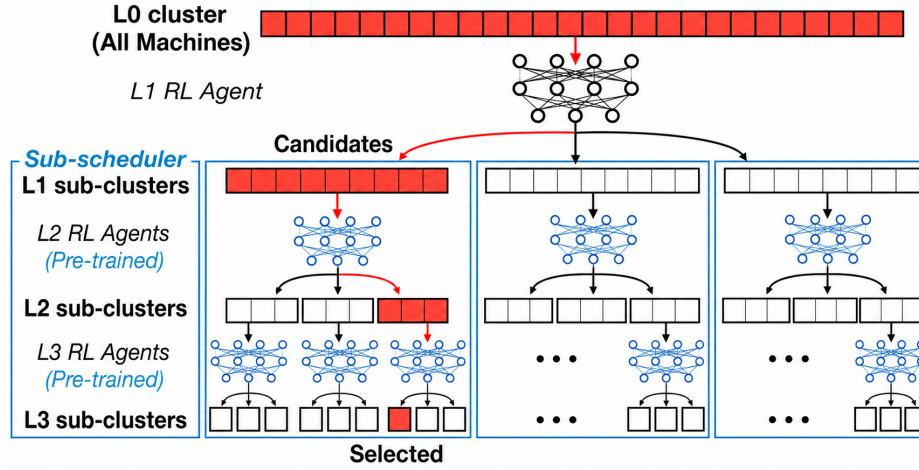


Figure 3.4: Hierarchical reinforcement-learning architecture for scalable container placement.

3.4.1 Dedicated On-the-spot Training

A distinctive choice in METIS is to train a dedicated model for each arriving container group rather than relying only on one universal offline policy. This may seem expensive, but it is justified by the LRA setting. LRAs are long-running; their execution lasts much longer than the scheduling decision. Spending extra time to find a high-quality placement can be worthwhile because the benefit persists for a long period.

The dedicated training strategy also reduces the diversity of inputs that a single policy must handle. Instead of learning a universal mapping for all possible container groups, the model specializes in the current group. This improves placement quality, but it also introduces a trade-off: scheduling latency increases. METIS relies on the observation that LRAs tolerate such latency better than short-lived batch jobs.

3.4.2 Experience Replay and Reward Design

Because RL training can have high variance, METIS uses experience replay. High-reward trajectories are stored and periodically replayed during policy updates. This technique allows the scheduler to learn repeatedly from good placements discovered earlier. The reward itself can be adapted to the operator’s goal. For example, if the goal is maximum throughput, the reward can be normalized request throughput. If the goal is SLO satisfaction, the reward can penalize SLO violations. This flexibility is a major advantage over fixed constraint-based schedulers.

However, reward flexibility is also a limitation. A reward can encourage SLO satisfaction, but it does not necessarily provide a hard guarantee. If violating SLOs temporarily helps maximize long-term reward, an RL scheduler may choose such behavior unless the reward is carefully shaped or constraints are enforced separately. This limitation becomes important when comparing METIS with TETRIS and OPTSCALER.

3.5 Overall System Architecture

The overall architecture of METIS contains four components. The input component receives and batches container launching requests. The scheduling component uses HRL and the simulator to explore placement decisions. The execution component launches containers according to the selected placement. The profiling component measures real performance and provides feedback for simulator refinement. These components are tied together by the same feedback path shown in Figure 3.3: profiling data improves the simulator, and the simulator supports subsequent scheduling decisions.

This architecture demonstrates the central design principle of METIS: replace hand-crafted placement knowledge with measured performance data. The scheduler does not need to know whether two applications should be co-located because of affinity or separated because of interference. It only needs to learn the performance consequence of different placements.

3.6 Evaluation and Results

3.6.1 Experimental Setup

The evaluation of METIS focuses on large-scale LRA placement. The system is implemented as a pluggable scheduling service and evaluated on Amazon EC2 clusters. The workloads include seven real-world applications covering machine learning, stream processing, I/O, and storage services. The baselines include constraint-based and heuristic schedulers such as Medea and Paragon [5, 6].

3.6.2 Throughput and Placement Quality

The main result is that METIS improves throughput compared with traditional constraint-based schedulers. In the EC2 experiment summarized in Figure 3.5, it achieves up to 25% higher average RPS than Medea and up to 61% higher average RPS than Paragon. This

outcome supports the central hypothesis: directly learning the performance impact of co-location can outperform manually specified qualitative constraints. The improvement is especially meaningful because it is obtained in a setting where the placement problem is both large and complex.

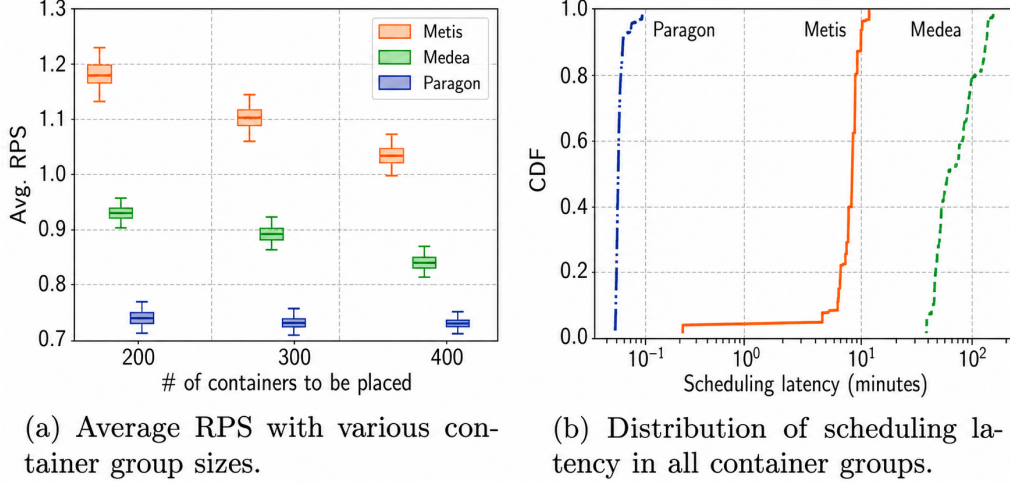


Figure 3.5: Performance comparison of METIS, Medea, and Paragon in an EC2 cluster.

Panel (a) of Figure 3.5 shows that the advantage of METIS is stable across container-group sizes. As the group size increases from 200 to 400, all methods see some degradation because the placement space becomes harder, but METIS consistently keeps the highest normalized RPS. The reason is not merely that METIS batches requests; Medea also reasons about groups. The difference is that Medea optimizes a qualitative constraint model, whereas METIS learns the quantitative performance consequence of co-location. Panel (b) exposes the cost of this learning-based decision: Paragon finishes in seconds, METIS takes minutes, and Medea can take much longer because it solves a large ILP. For LRAs, this latency is acceptable because the placement benefit persists over a long execution period.

3.6.3 Scalability of Hierarchical Learning

The second evaluation dimension is scalability. A flat RL scheduler does not scale because both state and action spaces grow with cluster size. HRL reduces this burden, and the scalability evidence in Figure 3.6 tests exactly this claim: whether hierarchical learning can still schedule thousands of containers and whether pretrained sub-schedulers reduce training cost. This result is important because without scalability, learning-based placement would be only a small-cluster demonstration rather than a practical scheduler.

In Figure 3.6(A), METIS is evaluated in a 729-node cluster with groups of 1000, 2000, and 3000 containers. Even at this scale, the RL agent reaches high-quality placements with normalized RPS roughly between 1.0 and 1.2, and the scheduling latency remains within about one hour. This matters because Medea-style ILP solving becomes impractical at this scale, while a flat RL model cannot explore the enormous action space effectively.

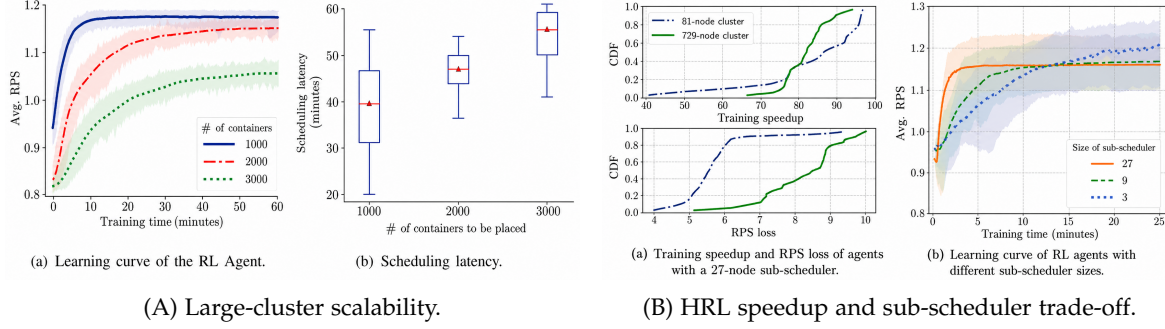


Figure 3.6: Scalability of hierarchical learning in METIS.

The companion result in Figure 3.6(B) explains where the scalability comes from: using a pretrained 27-node sub-scheduler provides roughly $40\times$ – $95\times$ training speedup with less than 10% RPS loss. Larger sub-schedulers learn faster because they reduce the depth and online search burden, but they also trade a small amount of placement quality for reusable offline knowledge. This is the central engineering trade-off in METIS: it spends offline training effort to make online hierarchical decisions feasible.

3.7 Summary and Limitations

METIS solves the first stage of cloud resource control: initial placement. Its strength is that it replaces qualitative manual constraints with quantitative performance-driven learning. It also addresses scalability through hierarchical decomposition. In the unified thesis narrative, METIS represents the transition from hand-crafted scheduling to data-driven scheduling.

Its limitations motivate the next chapter. First, it mainly focuses on initial placement. Once LRAs run for a long time, workload changes may make the initial placement suboptimal. Second, SLO satisfaction is handled through rewards or objectives rather than explicit hard constraints. Third, the system does not directly model migration cost or prediction uncertainty. These limitations motivate TETRIS, which studies runtime rescheduling with explicit migration cost and hard capacity constraints.

Chapter 4

Proactive Rescheduling and Migration: Tetris

4.1 Motivation

METIS answers where newly arriving containers should be placed. However, LRAs are long-running and time-varying. A placement that works well at time t may become imbalanced at time $t + 1$ or $t + 2$. Therefore, a cloud system also needs runtime rescheduling. The central action in runtime rescheduling is migration: moving a container from one server to another.

Migration is useful but costly. It can reduce load imbalance by moving workloads away from hot servers, but it can also disrupt service. In practice, migration may require stopping the container, copying memory state, restoring the container on another server, and restarting service. If a scheduler migrates a container only to move it back soon afterward, the system pays the migration cost without gaining stable balance. TETRIS calls this pattern invalid migration, and the back-and-forth movement in Figure 2.2 is the concrete failure mode the chapter tries to avoid.

The root cause of invalid migration is short-sighted optimization. A reactive scheduler may see that server s_1 is overloaded and move the largest container to server s_2 . But if s_2 becomes overloaded in the next time slot, the same container may need to move again. Such behavior is especially harmful for LRAs because they are stateful or partially stateful and have strict SLOs.

TETRIS proposes to solve this problem through MPC. MPC is a compromise between one-step reactive control and infinite-horizon optimization. It predicts resource consumption over a finite future window, optimizes actions over that window, executes only the first action, and repeats the process at the next time slot. The contrast in Figure 4.1 is useful here: TETRIS reasons over a bounded window and then replans, whereas the Metis+-style RL baseline learns a longer-horizon policy that is harder to keep aligned with short-term capacity constraints. This allows the scheduler to reason about near-future consequences without relying on long-range predictions that become unreliable.

4.2 Discrete-time Dynamic Model

TETRIS models the cluster as a discrete-time system. Let $\mathcal{M}$ be the set of servers and $\mathcal{N}$ be the set of containers. Time is slotted: $t \in \{t_0, t_1, \dots, t_W\}$, where W is the window size. For each container $k \in \mathcal{N}$, its CPU and memory consumption at time t are denoted as $\text{cpu}^k(t)$

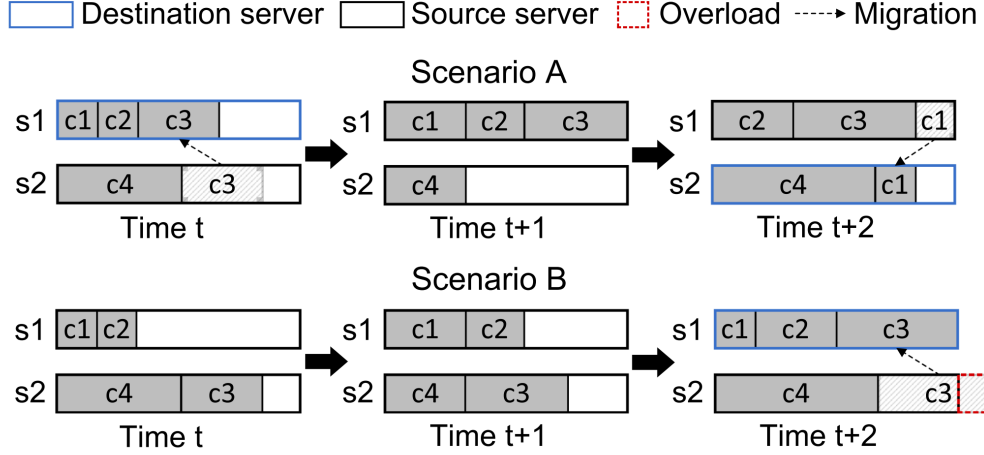


Figure 4.1: Illustrative example comparing MPC-based TETRIS and RL-based METIS+.

and $\text{mem}^k(t)$. The binary variable $x_i^k(t)$ indicates whether container k runs on server i :

$$x_i^k(t) = \begin{cases} 1, & \text{if container } k \text{ runs on server } i \text{ at time } t, \\ 0, & \text{otherwise.} \end{cases} \quad (4.1)$$

The first part of the objective is load imbalance. TETRIS uses the variance of resource consumption across servers. For CPU and memory, the load imbalance cost can be written as

$$C_b(t) = \frac{1}{|\mathcal{M}| - 1} \sum_{i \in \mathcal{M}} \left[\left(\sum_{k \in \mathcal{N}} x_i^k(t) \text{cpu}^k(t) - \overline{\text{CPU}}(t) \right)^2 + \beta \left(\sum_{k \in \mathcal{N}} x_i^k(t) \text{mem}^k(t) - \overline{\text{MEM}}(t) \right)^2 \right], \quad (4.2)$$

where $\overline{\text{CPU}}(t)$ and $\overline{\text{MEM}}(t)$ denote average CPU and memory usage across servers at time t , and β normalizes memory relative to CPU. A larger $C_b(t)$ indicates a more imbalanced cluster.

The second part is migration cost. Let $m^k(t)$ indicate whether container k is migrated at time t . The total migration cost is

$$C_m(t) = \sum_{k \in \mathcal{N}} C_{\text{mig}}^k(t) m^k(t), \quad (4.3)$$

where the unit migration cost is modeled as approximately linear in memory footprint:

$$C_{\text{mig}}^k(t) = \delta + \gamma \cdot \text{mem}^k(t). \quad (4.4)$$

Here δ is a fixed migration overhead and γ is the memory-dependent migration coefficient. The total per-slot scheduling cost is

$$C(t) = C_b(t) + \alpha C_m(t), \quad (4.5)$$

where α balances load balancing against migration cost.

4.3 Container Resource Prediction

MPC requires future resource estimates. TETRIS therefore includes a container resource predictor. TETRIS observes that containers exhibit different temporal patterns: the examples in Figure 4.2 include both repeated periodic cycles and irregular aperiodic fluctuations. This heterogeneity motivates the predictor split. TETRIS first categorizes containers and then applies different prediction models: *ARIMA* for periodic workloads and *LSTM* for aperiodic workloads [1, 7].

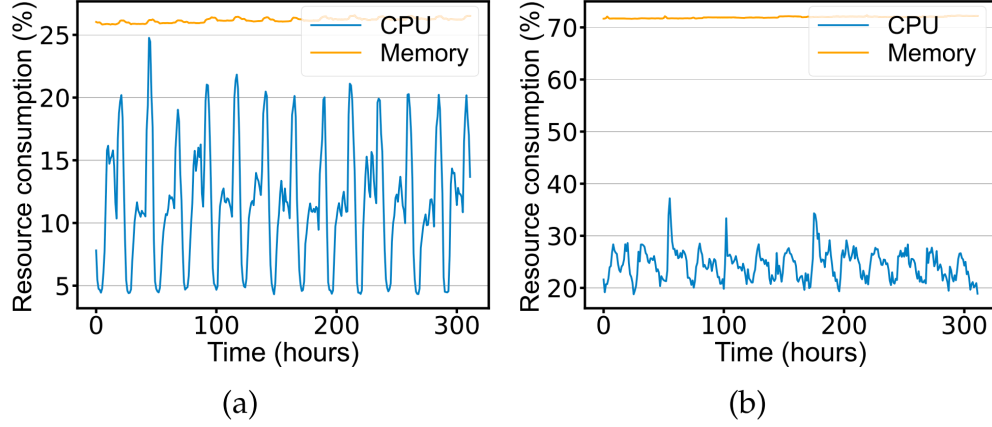


Figure 4.2: Examples of periodic and aperiodic container resource-consumption patterns.

This design is practical because no single simple predictor works best for all containers. Periodic workloads benefit from statistical time-series modeling, while aperiodic workloads require more flexible sequence models. The prediction horizon must also be chosen carefully. A longer horizon gives the scheduler more future awareness, but prediction error increases with horizon length. This is one reason TETRIS uses a finite sliding window rather than infinite-future optimization.

4.4 MPC Formulation for Migration-aware Scheduling

At each time slot, TETRIS solves an optimization problem over the window $[t_1, t_W]$:

$$\min_{x_i^k(t)} \sum_{t=t_1}^{t_W} C(t), \quad (4.6)$$

subject to placement and capacity constraints. Each container must run on exactly one server:

$$\sum_{i \in \mathcal{M}} x_i^k(t) = 1, \quad \forall k \in \mathcal{N}. \quad (4.7)$$

CPU and memory consumption on each server cannot exceed capacity:

$$\sum_{k \in \mathcal{N}} x_i^k(t) \text{cpu}^k(t) \leq \text{CPU}_i^{\text{cap}}(t), \quad \forall i \in \mathcal{M}, \quad (4.8)$$

$$\sum_{k \in \mathcal{N}} x_i^k(t) \text{mem}^k(t) \leq \text{MEM}_i^{\text{cap}}(t), \quad \forall i \in \mathcal{M}. \quad (4.9)$$

These constraints distinguish TETRIS from reward-only RL scheduling. A reward can penalize overload, but a hard constraint forbids it in the optimization model. This is why TETRIS provides a stronger SLO story than METIS. In the simplified SLO interpretation, containers hosted on overloaded servers are considered to experience SLO violations. By enforcing CPU and memory capacity constraints, the scheduler prevents such overload within the modeled horizon.

4.5 Efficient Solving through Candidate Filtering and Monte Carlo Sampling

Solving the full mixed-integer optimization problem for a large cluster is computationally expensive. TETRIS therefore uses two acceleration techniques. First, it restricts migration candidates using utilization thresholds. Servers with high utilization are candidate sources; servers with low utilization are candidate destinations. This reduces the number of container-server pairs that the optimizer must consider.

Second, TETRIS uses a Monte Carlo method to sample possible scheduling actions. Instead of enumerating all placements, the scheduler samples candidate migration plans, evaluates their projected cost over the MPC window, and selects the best sampled plan. This produces a practical sub-optimal solution. The method is appropriate because the goal is not to solve a mathematically exact offline optimization problem, but to make repeated online decisions in a production-like environment.

The complete pipeline in Figure 4.3 follows directly from these design choices: prediction supplies the finite-horizon demand estimates, the MPC scheduler searches over candidate migrations, and the migration module executes the selected first action before the next control step.

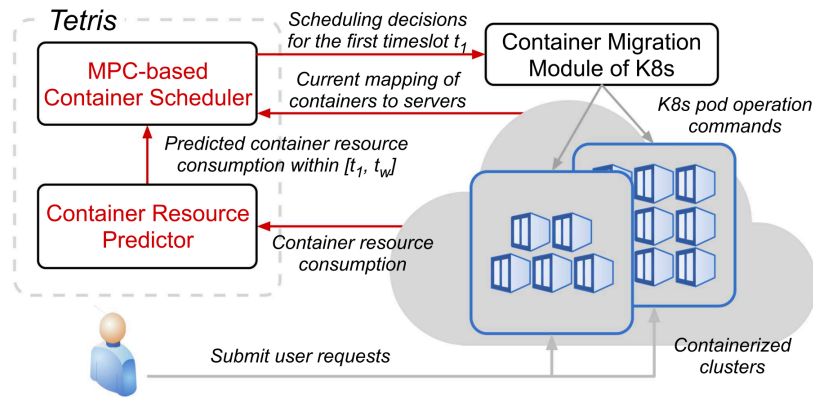


Figure 4.3: Overall architecture of TETRIS, including resource prediction, MPC-based scheduler, and container migration module.

Algorithm 1 Monte Carlo MPC solver in TETRIS.

Require: placement X_t , prediction $\hat{D}_{t:t+W}$, thresholds θ_h, θ_l , samples Z

Ensure: migration u_t

▷ filter phase

```
1:  $S_h \leftarrow \emptyset, S_l \leftarrow \emptyset$ 
2: for all server  $i \in \mathcal{M}$  do
3:   if  $u_i > \theta_h$  then
4:      $S_h.add(i)$ 
5:   else if  $u_i < \theta_l$  then
6:      $S_l.add(i)$ 
7:   end if
8: end for
```

▷ sample phase

```
9:  $P^* \leftarrow \emptyset, J^* \leftarrow \infty$ 
10: for  $z = 1$  to  $Z$  do
11:    $P_z \leftarrow \text{samplePlan}(X_t, S_h, S_l)$ 
12:    $J_z \leftarrow \sum_{\tau=t}^{t+W} C(P_z, \hat{D}_\tau)$ 
13:   if  $J_z < J^*$  then
14:      $P^* \leftarrow P_z, J^* \leftarrow J_z$ 
15:   end if
16: end for
```

▷ execute phase

```
17:  $u_t \leftarrow \text{first}(P^*)$ 
18: return  $u_t$ 
```

4.6 Evaluation and Results

4.6.1 Experimental Setup

TETRIS is evaluated using both a Kubernetes-based prototype and trace-driven simulations. The prototype runs on 10 Amazon EC2 instances and hosts 60 containers. The workloads include Apache Tomcat, Redis, and ResNet50. The baselines are Sandpiper, a greedy worst-fit load-balancing scheduler [16], and Metis+, an RL-based scheduling baseline inspired by METIS.

4.6.2 Load Balancing and Migration Cost

The evaluation shows that TETRIS improves load balancing while reducing migration cost, and Figure 4.4 is organized around this two-objective claim. This joint improvement is not trivial because the objectives can conflict. Moving many containers may quickly balance load but cause high migration cost. Moving few containers may reduce migration cost but leave the cluster imbalanced. The weighted objective $C_b(t) + \alpha C_m(t)$ explicitly captures this trade-off, so the key question is whether the curves improve both terms at the same time.

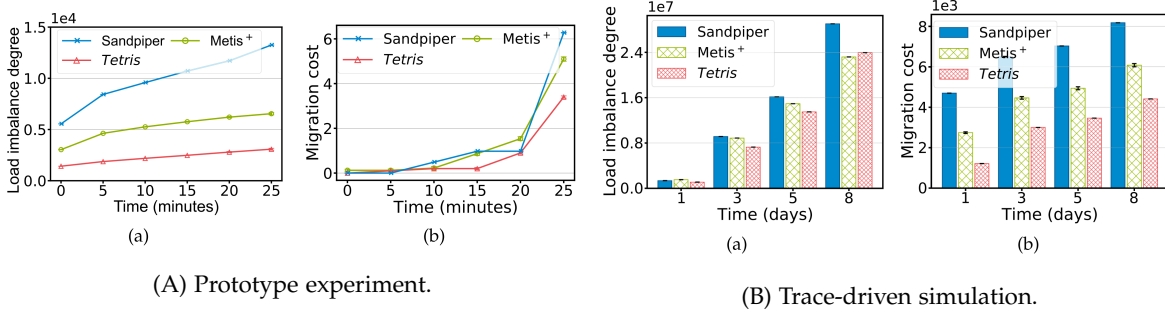


Figure 4.4: Performance comparison of TETRIS, Sandpiper, and Metis+ in terms of cumulative load imbalance and migration cost.

In the Kubernetes prototype result in Figure 4.4(A), TETRIS keeps both cumulative load imbalance and migration cost below Sandpiper and Metis+ on a 60-container cluster with 5-minute time slots. Quantitatively, it reduces load imbalance by 74.4%–77.8% compared with Sandpiper and by 53.0%–59.6% compared with Metis+. It also reduces migration cost by 7.8%–79.5% and 10.3%–77.0%, respectively. The important point is not merely that one metric improves, but that both improve together because the MPC objective optimizes load imbalance and migration cost over the same finite window.

The larger Alibaba-trace simulation in Figure 4.4(B) shows the same qualitative behavior over eight days. TETRIS reduces load imbalance by 9.7%–28.6% and migration cost by 27.4%–74.1% relative to the two baselines. The gap is smaller than in the prototype because the trace is larger and more variable, but the trend is consistent. Sandpiper accumulates high migration cost because it greedily reacts to current hotspots, while Metis+ eventually improves after online learning converges but performs worse in the early stage. The finite-horizon MPC formulation gives TETRIS a more stable online behavior.

4.6.3 Invalid Migration Reduction

A central claim of TETRIS is that finite-horizon MPC reduces invalid migrations. The intuition is clear: if the scheduler can see that a server will become overloaded in the next few slots, it avoids moving a container there now. Figure 4.5 tests this mechanism directly by plotting the cumulative invalid-migration count over time for TETRIS, Sandpiper, and Metis+.

The curve in Figure 4.5 explains why the aggregate migration cost decreases. Over the 24-hour Alibaba trace, Sandpiper accumulates invalid migrations quickly because it has no lookahead, especially when resource consumption changes sharply around hour 20. Metis+ performs better than Sandpiper because it includes a long-term objective, but its infinite-horizon RL formulation still permits temporary overload and repeated movement while the policy improves. TETRIS has the lowest invalid-migration count, reducing invalid migrations by 59.4%–81.3% compared with Sandpiper and by 32.4%–78.9% compared with Metis+. This result supports the main design claim: a finite, repeatedly re-optimized prediction window is a better fit for LRA migration than either one-step reaction or uncon-

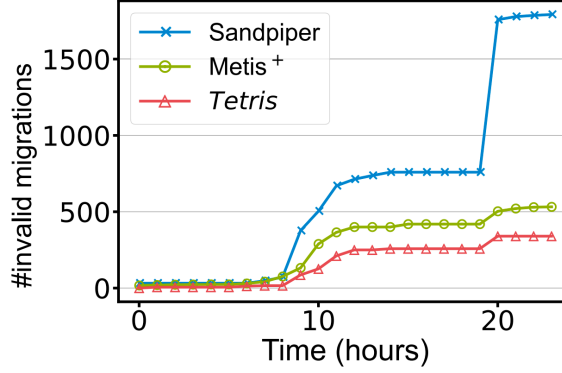


Figure 4.5: Cumulative number of invalid migrations under TETRIS, Sandpiper, and Metis+.

strained long-horizon learning.

4.6.4 SLO Violations and Runtime Overhead

The evaluation also considers SLO violations. In the prototype experiments, TETRIS causes zero SLO violations under the overload-based definition, while Sandpiper and Metis+ cause up to 35 and 12 violations within one time slot. In the large-scale simulation, TETRIS again has zero SLO violations over eight days, while Sandpiper and Metis+ can cause up to 14,403 and 918 violations, respectively. This supports the value of hard capacity constraints in the MPC formulation: overload is not only penalized, but forbidden by the CPU and memory constraints used during optimization.

Runtime overhead is another important metric. A scheduler that produces good plans too slowly is not practical. TETRIS reduces overhead through candidate filtering and sampling. Its complexity is much lower than exhaustive optimization and more predictable than online RL training.

4.7 Summary and Limitations

TETRIS extends the control story from initial placement to runtime rescheduling. It shows that long-running workloads require proactive migration-aware scheduling and that MPC is a natural method for this problem. Compared with METIS, TETRIS provides stronger SLO protection because it encodes CPU and memory capacities as hard constraints.

However, TETRIS still has limitations. Its predictions are deterministic. If the predicted CPU or memory consumption deviates significantly from reality, the optimizer may make unsafe decisions. It also focuses on migration rather than cluster-level capacity scaling. If total workload grows beyond the capacity of the existing cluster, migration alone is insufficient. These limitations motivate OPTSCALER, which treats uncertainty explicitly and controls the number of provisioned nodes.

Chapter 5

Robust Collaborative Autoscaling: OptScaler

5.1 Motivation: From Rescheduling to Autoscaling under Uncertainty

TETRIS assumes that the cluster has enough capacity and focuses on moving containers to balance load. But in many cloud settings, total workload changes over time. When global demand increases, the system may need more nodes; when demand decreases, keeping the same number of nodes wastes resources. Autoscaling addresses this problem by automatically adjusting resource capacity.

Autoscaling can be reactive, proactive, hybrid, or collaborative. Reactive autoscalers use current metrics and scale after overload is observed. Their advantage is simplicity, but their response is delayed. Proactive autoscalers predict future workload and scale in advance. Their advantage is anticipation, but they are vulnerable to prediction error. Hybrid autoscalers combine both, but if the two modules independently produce decisions, they may conflict. OPTSCALER proposes a collaborative approach: proactive prediction and reactive estimation are integrated through a shared optimization module.

Uncertainty is the key reason for collaboration. Workload forecasts are imperfect, especially under bursts. CPU-utilization metrics are noisy. Co-located LRAs can interfere with each other, changing the relationship between workload and CPU consumption. Hardware and cloud operations also impose constraints on scaling speed: adding or removing nodes takes time, and systems may limit how many nodes can be changed in one interval. OPTSCALER explicitly models these practical issues through MPC, chance constraints, and online feedback.

5.2 Cloud Deployment and Workload Characteristics

OPTSCALER focuses on horizontal autoscaling for co-located LRAs. In its deployment model, a cluster contains a variable number of nodes, and each node has equalized resource capacity. Each container hosts one LRA, and the workload of an LRA is distributed across host containers. In a balanced deployment, each node runs the same combination of LRAs, so adding or removing nodes changes the per-node workload.

The scaling metric is CPU utilization. The operational goal is to keep CPU utilization close to, but below, a desired threshold. If utilization is too high, SLOs may be violated. If utilization is too low, resources are wasted. This setup turns autoscaling into a control problem: choose the number of nodes so that future CPU utilization tracks a target under

workload uncertainty. The workload traces and CPU relation in Figure 5.1 motivate this model: demand contains repeated structure, yet the controller ultimately acts through the workload-to-utilization mapping. In the notation used by OPTSCALER, y_t denotes the workload vector at time t , x_t denotes the number of installed nodes, c_t denotes CPU utilization, and c^* denotes the desired CPU-utilization target.

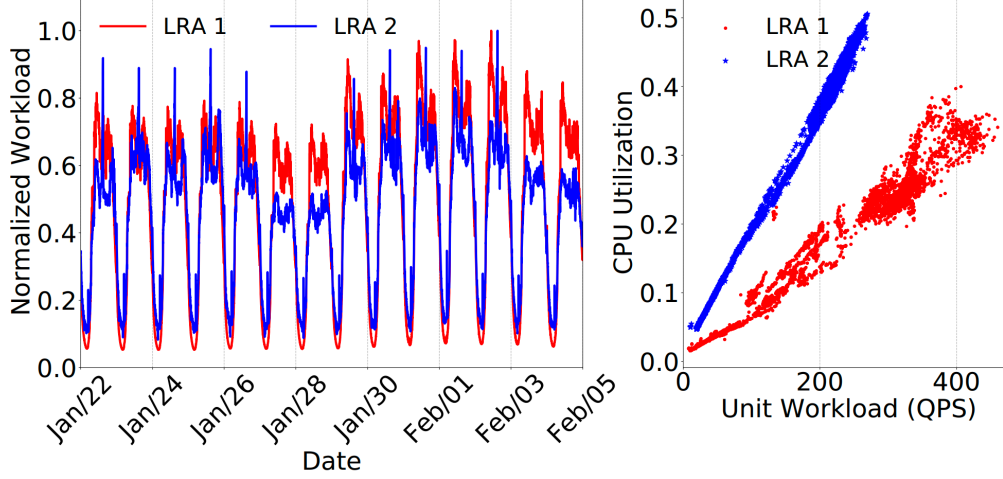


Figure 5.1: Periodic workload patterns and the linear relationship between unit workload and CPU utilization.

5.3 Framework Overview

OPTSCALER consists of three modules. The proactive module predicts future workloads for all LRAs. The reactive module estimates CPU utilization from unit workload and updates its parameters using real-time feedback. The optimization module receives inputs from both modules and solves an MPC problem with chance constraints. The output is a scaling plan, which is applied to the cloud system. The architecture in Figure 5.2 is important because the proactive and reactive modules do not issue independent scaling commands; both feed the same optimizer.

This architecture addresses the limitation of independent hybrid autoscalers. Instead of selecting either a proactive or reactive decision, OPTSCALER lets the proactive module provide future workload inputs and the reactive module provide a current workload-to-CPU mapping. The optimization module then makes one holistic decision. This is why the framework is called collaborative.

5.4 Proactive Workload Prediction

The proactive module predicts future workloads over a horizon of H time steps. The challenge is that LRAs can have both long-term periodic patterns and short-term local variations. A payment or recommendation service may show daily and weekly seasonality, but it may also experience local bursts. OPTSCALER therefore decomposes prediction into a

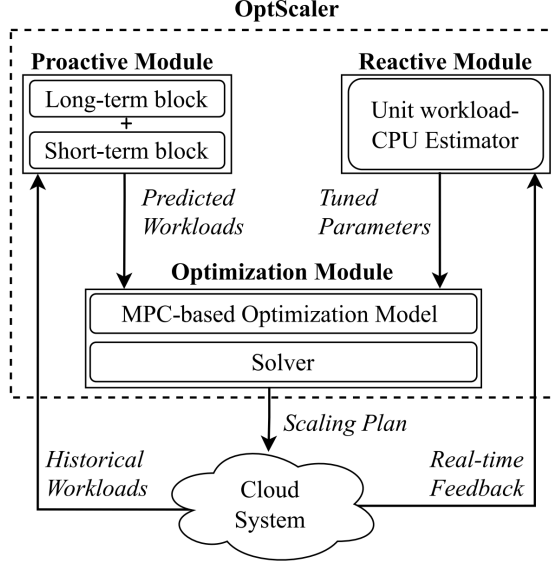


Figure 5.2: Overall architecture of OPTSCALER: proactive module, reactive module, optimization module, cloud system, and feedback.

long-term periodic block and a short-term local block.

Let y_n^t denote the workload of LRA n at time t . The prediction task is to estimate

$$\mathbf{y}_n^{t+1:t+H} = [y_n^{t+1}, \dots, y_n^{t+H}] \quad (5.1)$$

from historical workloads and known covariates:

$$\hat{\mathbf{y}}_n^{t+1:t+H} = F_{\Theta}(\mathbf{y}_n^{t-C:t}, \mathbf{z}_n^{t-C:t+H}, n), \quad (5.2)$$

where $\hat{\mathbf{y}}$ denotes predicted workload, F_{Θ} is the prediction model with parameters Θ , C is the look-back window, $\mathbf{z}$ may include time and calendar covariates, and n identifies the LRA.

5.4.1 Long-term Periodic Block

The long-term periodic block uses Fourier series to represent periodic workload patterns:

$$\hat{y}_p^t = \sum_{r=0}^R \left(a_r \cos \frac{2\pi r t}{P} + b_r \sin \frac{2\pi r t}{P} \right), \quad (5.3)$$

where P is the period and R is the truncation order. Fourier representation is suitable because many cloud workloads have daily or weekly periodicity. It avoids feeding extremely long historical sequences into a neural model.

5.4.2 Short-term Local Block

The short-term local block models the residual between actual workload and long-term periodic estimate. It uses a sequence-to-sequence structure with Flow-Attention. The purpose is to capture local trends, deviations, and bursts that cannot be explained by periodicity

alone. Figure 5.3 makes this decomposition explicit: a long-term periodic block captures stable seasonal structure, while a local block learns the remaining short-term variation. The final prediction is the sum of the periodic and local components:

$$\hat{\mathbf{y}}^{t+1:t+H} = \hat{\mathbf{y}}_p^{t+1:t+H} + \hat{\mathbf{y}}_l^{t+1:t+H}. \quad (5.4)$$

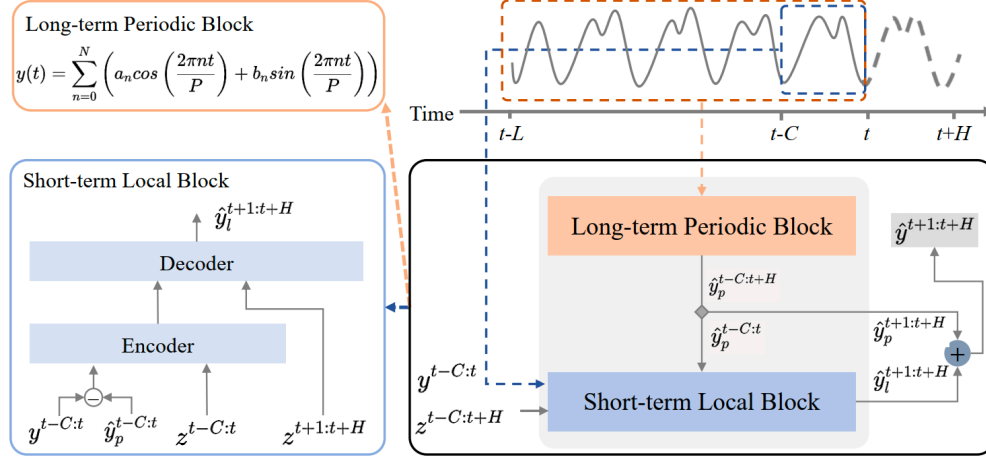


Figure 5.3: Workload prediction model in the proactive module, including long-term periodic block and short-term local block.

5.4.3 Comparison with Tetris Prediction

TETRIS predicts container-level resource consumption and trains dedicated predictors for containers. OPTSCALER instead predicts workloads for LRAs using a unified model. This difference reflects the control object. TETRIS needs CPU/memory predictions for migration decisions. OPTSCALER needs workload predictions to decide node counts. The unified predictor is more appropriate for production autoscaling because many LRAs must be handled together.

5.5 Reactive CPU Estimator and Feedback Adaptation

The reactive module estimates CPU utilization from workload and node count. If $\mathbf{y} \in \mathbb{R}^N$ is the workload vector for N LRAs and x is the number of nodes, then the unit workload is $\mathbf{y}/x$. OPTSCALER models CPU utilization as

$$c = f\left(\frac{\mathbf{y}}{x}\right) = w_b + \frac{\mathbf{y}^\top \mathbf{w}_k}{x} + \epsilon, \quad (5.5)$$

where w_b is the base CPU overhead, $\mathbf{w}_k$ captures the CPU contribution of each LRA, and ϵ is an uncertainty term.

The estimator is updated using real-time feedback. If $\hat{c}_t$ is the estimated CPU utilization and c_t is the observed CPU utilization, the error is $e_t = \hat{c}_t - c_t$. With feedback strength η , a Widrow-Hoff-style update can be written as

$$\mathbf{w}_{k,t} = \mathbf{w}_{k,t-1} - \eta e_t \frac{\mathbf{y}_t}{x_t}. \quad (5.6)$$

The feedback strength controls how aggressively the estimator adapts. A large value can reduce SLO violations by reacting quickly, but it may also over-provision resources. A small value reduces resource cost but may respond too slowly to system drift.

5.6 Robust MPC with Chance Constraints

The optimization module chooses scaling decisions over the future horizon H . Using weights q and r to balance CPU-target tracking and resource cost, a simplified objective is

$$\min_{x_{t+1:t+H}} \sum_{h=1}^H [q(c_{t+h} - c^*)^2 + rx_{t+h}], \quad (5.7)$$

where x_{t+h} is the number of nodes at future step $t+h$.

The constraints include practical scaling limits. Let $x_{\min}$ and $x_{\max}$ be the lower and upper bounds on node count, and let $\Delta_{\max}$ be the maximum node-count change allowed in one scaling interval:

$$x_{\min} \leq x_{t+h} \leq x_{\max}, \quad (5.8)$$

$$|x_{t+h} - x_{t+h-1}| \leq \Delta_{\max}. \quad (5.9)$$

The first constraint keeps node count within safe bounds. The second captures limited scaling speed: a cloud system cannot add or remove arbitrary numbers of nodes instantly.

The robust part comes from chance constraints. Instead of assuming the CPU estimate is exact, OPTSCALER treats it as uncertain. With ρ denoting the acceptable risk level, the SLO-related constraint is required to hold with high probability:

$$\Pr(c_{t+h} \leq c^*) \geq 1 - \rho, \quad (5.10)$$

This formulation explicitly acknowledges prediction error and metric noise. It also provides a conceptual improvement over deterministic MPC: the controller is not merely optimizing under one predicted trajectory, but protecting against a distribution of possible outcomes.

5.7 Evaluation and Results

5.7.1 Prediction Accuracy

The first evaluation question is whether the proactive module predicts workloads accurately. OPTSCALER compares its prediction model against prevalent methods because autoscaling decisions depend on future workload estimates, especially near interval peaks. The ECDFs in Figure 5.4 show the distribution of normalized absolute prediction error, while Table 5.1 records the exact WAPE values for minute-level and interval-peak prediction.

The values in Table 5.1 show that prediction difficulty differs across workload sets. All models perform better on $\mathcal{S}_A$, whose workloads are more regular, and worse on $\mathcal{S}_B$ and $\mathcal{S}_C$, where local fluctuations are stronger. OPTSCALER obtains the best minute-level WAPE

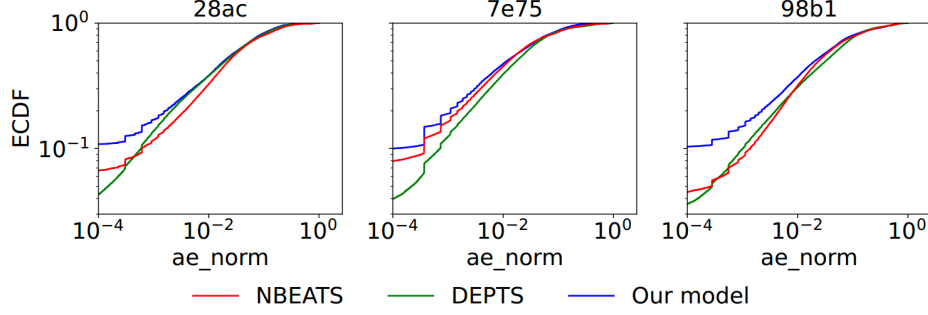


Figure 5.4: Workload prediction performance of OPTSCALER compared with baseline predictors.

Table 5.1: Workload prediction accuracy in OPTSCALER. Lower WAPE is better.

Model	Minute-level WAPE			Interval-peak WAPE		
	$\mathcal{S}_A$	$\mathcal{S}_B$	$\mathcal{S}_C$	$\mathcal{S}_A$	$\mathcal{S}_B$	$\mathcal{S}_C$
ARIMA	0.087	0.280	0.432	0.107	0.286	0.420
Autoformer	0.079	0.258	0.467	0.085	0.274	0.454
DEPTS	0.066	0.243	0.356	0.082	0.257	0.342
NBEATS	0.067	0.218	0.352	0.085	0.178	0.377
OPTSCALER	0.063	0.224	0.309	0.080	0.171	0.341

on $\mathcal{S}_A$ and $\mathcal{S}_C$, and it is close to the best neural baseline on $\mathcal{S}_B$. More importantly for autoscaling, it obtains the best interval-peak WAPE on all three sets. This matters because the optimization module uses interval peak demand to avoid overload; a predictor that is slightly better on average but worse at peaks may still produce unsafe scaling decisions. The ECDF curves in Figure 5.4 provide the complementary distributional view: OPTSCALER places more probability mass at smaller normalized absolute errors for the three difficult LRAs in $\mathcal{S}_C$, showing that the improvement is not only an artifact of an aggregate average.

5.7.2 Autoscaling Framework Comparison

The second evaluation question is whether OPTSCALER makes better scaling decisions than reactive, proactive, and hybrid baselines. The metrics include SLO violation rate S_{vr} , accumulated violation magnitude V_{sum} , and average resource cost R_{avg} ; the baseline set includes representative autoscaling approaches such as Autopilot [11]. The aggregate comparison in Table 5.2 measures the safety-cost trade-off, and the CPU-utilization trajectories in Figure 5.5 show how those aggregate numbers arise over time. Together, they test whether collaborative optimization improves SLO satisfaction without simply over-provisioning.

Table 5.2 makes the safety-cost trade-off explicit. Autopilot becomes safer when its percentile parameter increases from 90% to 95%, but the additional conservatism can sharply increase resource cost, especially on $\mathcal{S}_B$. Madu benefits from workload prediction but has no reactive correction, so forecast error appears directly as SLO risk. HAS reduces this risk by combining proactive and reactive decisions, but the two components still remain

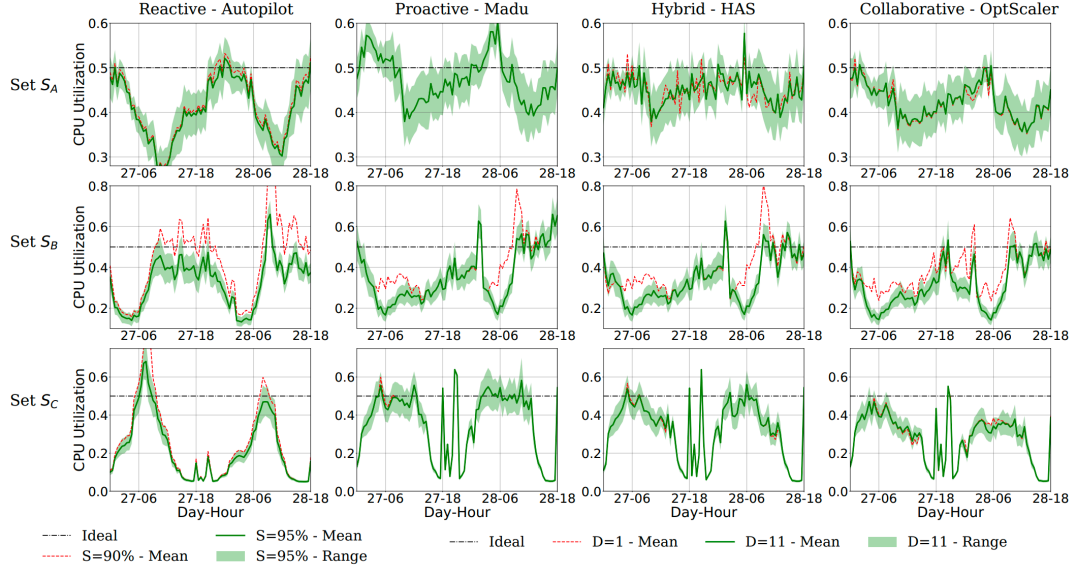


Figure 5.5: CPU utilization comparison across autoscaling frameworks.

Table 5.2: Autoscaling comparison in OPTSCALER. Lower values are better.

Type	Name	Param	S_A			S_B			S_C		
			S_{vr} (%)	V_{sum}	R_{avg}	S_{vr} (%)	V_{sum}	R_{avg}	S_{vr} (%)	V_{sum}	R_{avg}
Reactive	Autopilot	$S = 90\%$	6.1	1.87	115.9	13.8	45.85	193.0	9.6	34.52	132.6
Reactive	Autopilot	$S = 95\%$	2.8	0.87	119.0	1.8	4.98	266.7	4.3	9.64	158.8
Proactive	Madu	$D = 1$	13.7	8.87	99.0	5.6	13.20	214.9	7.0	8.74	69.5
Proactive	Madu	$D = 11$	13.6	8.65	99.1	2.7	3.22	252.5	6.5	8.34	70.0
Hybrid	HAS	$D = 1$	1.6	0.73	105.1	5.3	13.96	223.0	3.6	3.36	81.3
Hybrid	HAS	$D = 11$	1.5	0.58	104.6	2.2	2.21	263.2	2.8	3.13	80.4
Collaborative	OPTSCALER	$D = 1$	1.2	0.26	113.0	2.1	3.56	226.3	1.1	0.63	84.8
Collaborative	OPTSCALER	$D = 11$	0.7	0.25	112.3	0.6	0.61	265.7	0.7	0.55	90.5

loosely coupled. OPTSCALER gives the best or near-best SLO metrics across all workload sets. Against HAS with $D = 11$, it reduces S_{vr} by 53%–75% and V_{sum} by 56%–82%, while increasing R_{avg} by only about 1%–12%. The trajectories in Figure 5.5 explain these numbers visually: reactive-only control fluctuates around the threshold, pure proactive control can miss workload spikes, and HAS still shows overshoots, while OPTSCALER keeps the CPU trajectory closer to the safe side of c^* .

The ablation study isolates the contribution of the collaborative autoscaling framework and the proposed workload predictor. The three configurations in Table 5.3 separate these two effects: Exp.1 uses HAS with NBEATS, Exp.2 switches only the framework to OPTSCALER, and Exp.3 uses the full OPTSCALER design.

The ablation results confirm that the robustness is not caused by a single component. Moving from Exp.1 to Exp.2 in Table 5.3 keeps the NBEATS predictor fixed but replaces HAS with the collaborative OPTSCALER framework; this alone reduces S_{vr} by roughly 27%–60%. Moving from Exp.2 to Exp.3 keeps the framework fixed but replaces NBEATS with our workload predictor; this further reduces S_{vr} by roughly 25%–53% and also reduces

Table 5.3: Ablation comparison between OPTSCALER and HAS with NBEATS. Lower values are better.

ID	Autoscaling Framework	Prediction Model	$\mathcal{S}_A$			$\mathcal{S}_B$			$\mathcal{S}_C$		
			S_{vr} (%)	V_{sum}	R_{avg}	S_{vr} (%)	V_{sum}	R_{avg}	S_{vr} (%)	V_{sum}	R_{avg}
Exp.1	HAS	NBEATS	1.1	0.60	105.9	2.0	3.04	254.3	3.7	8.75	78.8
Exp.2	OPTSCALER	NBEATS	0.8	0.40	114.5	0.8	0.76	277.6	1.5	5.32	133.8
Exp.3	OPTSCALER	Our model	0.7	0.25	112.3	0.6	0.61	265.7	0.7	0.55	90.5

violation magnitude. Overall, the full design achieves 36%–81% fewer SLO violations than the strongest hybrid baseline, at a 4%–15% resource-cost increase. The prediction model improves the quality of the forecast, while the collaborative MPC framework turns that forecast into safer scaling decisions.

5.7.3 Sensitivity and Robustness

The feedback strength η exposes a resource-cost versus SLO-safety trade-off. Larger η makes the reactive estimator more aggressive, which can suppress SLO violations but may lead to over-provisioning. Smaller η is more conservative in resource cost but may under-react to deviations. The sensitivity curve in Figure 5.6 is therefore not just a parameter sweep; it shows that production deployment requires policy tuning according to local SLO and budget priorities.

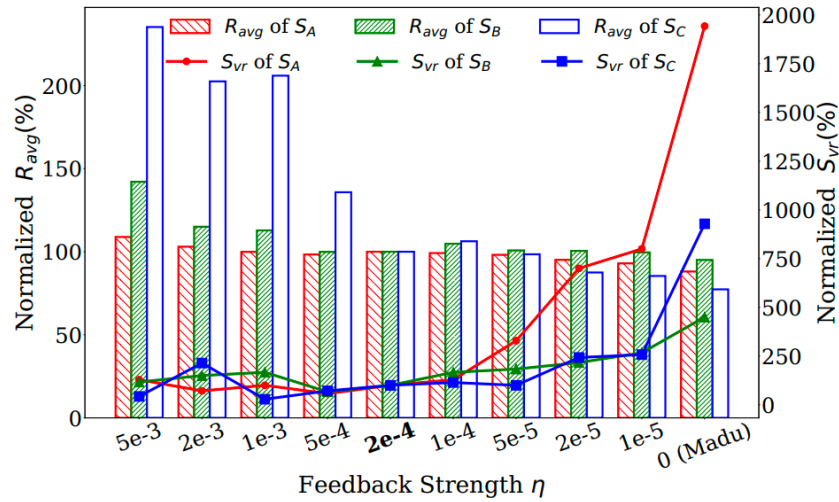


Figure 5.6: Sensitivity of feedback strength η , showing the trade-off between resource cost and SLO violation rate.

When η is large in Figure 5.6, the estimator reacts aggressively to CPU-utilization error, which suppresses SLO violations but can over-provision nodes. When η approaches zero, the reactive module is effectively disabled and the behavior approaches a pure proactive scaler, reducing resource cost but increasing violation risk. The curves show that R_{avg} and S_{vr} move in opposite directions as η changes. A practical operating point lies around

5×10^{-4} to 10^{-4} , close to the default 2×10^{-4} used in the main experiments. This parameter therefore exposes a clear policy knob: operators can tune it according to whether the local priority is stricter SLO protection or lower resource cost.

5.7.4 Online Deployment at Alipay

A major strength of OPTSCALER is online deployment evidence. The system has been deployed at Alipay to support autoscaling for online LRAs. The installed-node count and CPU-utilization trace in Figure 5.7 provide the key production test: the controller must track demand changes while keeping utilization in a desirable range. This deployment evidence distinguishes OPTSCALER from purely experimental prototypes and demonstrates that collaborative robust autoscaling is practical in a production environment.

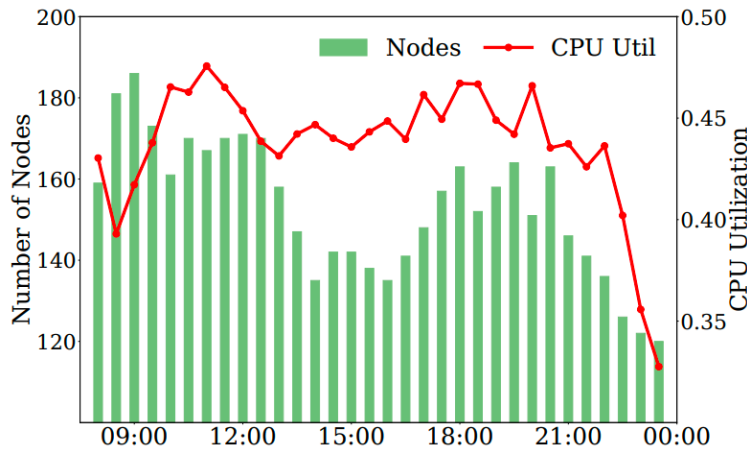


Figure 5.7: Online autoscaling result at Alipay, showing installed nodes and CPU utilization over time.

The online curve in Figure 5.7 shows that the offline conclusions carry over to deployment. During the daytime experiment, CPU utilization stays in a narrow desirable band, roughly between 0.40 and 0.48, while the number of installed nodes follows the workload trend. After 10 PM, workload drops and the controller correspondingly reduces provisioned capacity. Compared with the previous fixed safe allocation of 190 nodes, the average resource cost under OPTSCALER is 152.9 nodes, giving a 19.5% resource saving while maintaining controlled CPU utilization. This result is important because it validates the collaborative design under production feedback, where prediction error, measurement noise, and operational scaling delays all occur together.

5.8 Summary and Limitations

OPTSCALER extends the thesis’s control trajectory to cluster-level autoscaling. It introduces uncertainty-aware decision making through reactive feedback and chance constraints. In the unified view, it strengthens the SLO story from deterministic constraints to probabilistic robust constraints.

Nevertheless, limitations remain. OPTSCALER assumes a relatively structured deployment where nodes host comparable combinations of LRAs. It focuses on horizontal scaling rather than placement or migration. It also relies on the quality of workload prediction and CPU-estimator assumptions. Most importantly for this thesis, OPTSCALER is not integrated with METIS or TETRIS. A complete cloud resource controller would need to combine placement, migration, and scaling decisions.

Chapter 6

Synthesis and Comparative Discussion

6.1 Evolution of Control Scope: Placement to Migration to Scaling

The three systems form a natural evolution in control scope. METIS controls the placement of newly arriving containers. TETRIS controls migration of already running containers. OPTSCALER controls the number of nodes or amount of cluster capacity. This progression mirrors the lifecycle of LRAs. A workload first arrives and must be placed; then it runs and may need migration; finally, aggregate demand may change and require autoscaling.

This control-scope view prevents a common misunderstanding. METIS, TETRIS, and OPTSCALER are not direct competitors solving the same problem. They solve adjacent problems at different levels. A production system could use all three: METIS for admission placement, TETRIS for periodic rescheduling, and OPTSCALER for capacity adjustment.

6.2 Evolution of Methodology: RL to MPC to Robust MPC

The methodology also evolves. METIS uses RL because the placement objective is difficult to write explicitly. It learns from performance data and simulator feedback. TETRIS uses MPC because runtime rescheduling has explicit costs and constraints: load imbalance, migration cost, CPU capacity, and memory capacity. OPTSCALER uses robust MPC because autoscaling must handle uncertain predictions and noisy feedback.

This methodological progression also changes how operators can reason about decisions. METIS produces a learned policy that is flexible but less transparent. TETRIS exposes its finite-horizon objective and capacity constraints, making the migration decision easier to audit. OPTSCALER further exposes the uncertainty model and risk level used by chance constraints. Thus, the thesis moves from learning hidden placement effects to optimizing explicit migration trade-offs and finally to controlling autoscaling risk under uncertainty.

6.3 Evolution of SLO Guarantees

The treatment of SLOs is one of the clearest progressions. In METIS, SLOs can be included in the reward, but the reward remains a soft objective. In TETRIS, resource capacities are hard constraints. If the model is correct, the optimizer should not choose placements that exceed CPU or memory capacity. In OPTSCALER, even deterministic constraints are not enough because predictions are uncertain. Chance constraints enforce that utilization remains below the threshold with high probability.

This does not mean the later systems simply dominate the earlier ones. A soft reward is useful when the system effect is hard to model, as in interference-aware placement. Deterministic constraints are appropriate when the controlled variables are directly observable and predictions are sufficiently reliable, as in short-horizon migration. Probabilistic constraints are necessary when the controller must act before the true demand is known, as in autoscaling. The correct SLO formulation therefore depends on the control layer.

6.4 Data, Prediction, and Feedback

All three systems are data-driven, but they use data differently. METIS uses data to learn placement-performance relationships. Its simulator maps co-location states to performance metrics. TETRIS uses time-series data to predict container resource consumption. Its feedback is mainly the repeated MPC loop: observe, predict, optimize, execute, and repeat. OPTSCALER uses historical workload for long-term prediction and real-time feedback for online CPU-estimator adaptation.

The prediction target shifts with the control problem. METIS predicts the performance effect of placement. TETRIS predicts future CPU and memory trajectories for migration planning. OPTSCALER predicts workload and estimates CPU utilization online. The main risk also shifts: simulator mismatch in METIS, horizon-dependent prediction error in TETRIS, and uncertainty-model mismatch in OPTSCALER.

6.5 System-level Trade-offs

The three systems also illustrate several system-level trade-offs.

6.5.1 Optimality versus Scalability

A fully optimal scheduler is often intractable. METIS avoids flat RL by hierarchical decomposition. TETRIS avoids full MILP solving by candidate filtering and Monte Carlo sampling. OPTSCALER uses structured optimization and simplified workload-to-CPU estimation to make robust autoscaling tractable. In all three cases, the system sacrifices exact global optimality for practical scalability.

6.5.2 Responsiveness versus Stability

Reactive systems respond quickly to observed metrics but can be unstable or delayed. Proactive systems are more stable if predictions are accurate, but they can fail under unexpected spikes. OPTSCALER explicitly combines both through collaboration. TETRIS also shows a related compromise: it avoids infinite-horizon optimization and instead uses a finite sliding window, improving stability over short-sighted decisions while avoiding unreliable long-range prediction.

6.5.3 Resource Cost versus SLO Safety

SLO safety often requires extra resources. OPTSCALER’s feedback-strength analysis makes this trade-off explicit: stronger feedback can suppress SLO violations but may over-provision.

This trade-off is present in all cloud resource control. A placement that maximizes utilization may increase interference; a migration that balances load may disrupt service; an autoscaler that prevents violations may waste resources.

6.5.4 Interpretability versus Black-box Learning

METIS is flexible but more black-box because its policy is learned. TETRIS and OPTSCALER are more interpretable because they expose objectives and constraints. In production cloud systems, interpretability is valuable for debugging and operator trust. This does not mean learning should be avoided; rather, learned components should be integrated with explicit control structure when possible.

6.6 Toward an Integrated Cloud Resource Controller

A natural future system would integrate the three control levels. When a new LRA arrives, a Metis-like placement module could choose an initial node. During execution, a Tetris-like module could periodically evaluate whether migration is necessary. In parallel, an OptScaler-like module could adjust the number of nodes based on aggregate workload and uncertainty. These modules should not operate independently. For example, autoscaling changes the available placement space; migration changes per-node utilization; placement affects future interference and scaling needs.

An integrated controller would require a shared state representation, common SLO definitions, and coordination among actions. It would also require conflict resolution. Scaling out may reduce the need for migration; migration may delay scaling; placement may avoid future migration. The unified feedback-control loop in Chapter 2 provides a starting abstraction, but a complete implementation remains open.

6.7 Limitations of This Synthesis

This synthesis has limitations. First, the three systems target different control levels and are evaluated under different platforms and workloads, so their quantitative results should not be read as a direct ranking. Second, each system simplifies part of the full cloud problem. METIS focuses on placement and does not directly enforce robust SLO constraints. TETRIS focuses on migration and assumes deterministic predictions. OPTSCALER focuses on autoscaling and assumes structured LRA deployment. Third, the thesis emphasizes CPU and memory because these resources are central in the three systems, but real clusters also face GPU, network, disk, and memory-bandwidth constraints. Finally, none of the three systems alone provides a complete end-to-end control plane for all cloud resource decisions.

These limitations do not weaken the unified story. Rather, they identify the next research direction: building a coordinated, multi-resource, uncertainty-aware cloud resource controller that combines learned placement, predictive migration, and robust autoscaling.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This thesis studied data-driven control for cloud resource orchestration through three representative systems: METIS, TETRIS, and OPTSCALER. The central argument is that these systems form a coherent progression rather than a collection of independent techniques.

METIS addresses the scale challenge in initial placement. It observes that hand-crafted placement constraints are qualitative and incomplete, and it replaces them with learned performance-driven scheduling. Its hierarchical reinforcement-learning design makes large-scale placement tractable.

TETRIS addresses the dynamics challenge in runtime rescheduling. It observes that time-varying LRAs can cause invalid migrations under short-sighted scheduling. By predicting resource consumption and applying MPC, it jointly optimizes load balancing and migration cost while enforcing hard capacity constraints.

OPTSCALER addresses the uncertainty challenge in autoscaling. It observes that reactive, proactive, and independent hybrid autoscalers are insufficient under workload fluctuations, metric noise, and co-location interference. By combining proactive prediction, reactive feedback, robust MPC, and chance constraints, it improves SLO robustness and resource efficiency.

Together, the three systems reveal a broader design trajectory. The control object evolves from container placement to container migration to cluster capacity. The methodology evolves from RL to MPC to robust MPC. The SLO treatment evolves from soft reward to deterministic constraints to probabilistic constraints. This trajectory suggests that future cloud operating systems should be understood not merely as resource allocators, but as data-driven feedback controllers operating under scale, dynamics, and uncertainty.

7.2 Future Work

7.2.1 Multi-resource Robust Control

The three systems primarily emphasize CPU and memory. Production clusters also require GPU, network bandwidth, disk I/O, cache capacity, and memory bandwidth management. Future systems should model multi-resource interference and enforce SLO constraints across all bottleneck resources. This extension is especially important for machine-learning workloads and data-intensive services.

7.2.2 Low-overhead Online Learning

Online learning improves adaptivity but introduces overhead. METIS must train policies; TETRIS must predict container resources and sample migration plans; OPTSCALER must update estimators and solve robust optimization problems. Future work should reduce this overhead through incremental learning, transfer learning, lightweight uncertainty estimation, and hardware-aware optimization.

7.2.3 Stronger Theoretical Guarantees

Data-driven controllers need stronger theoretical foundations. Useful guarantees may include stability bounds, regret bounds, probabilistic SLO guarantees, and worst-case resource-cost bounds. A promising direction is to combine learning-based prediction with formal constrained optimization, so that learned models improve efficiency while safety constraints remain explicit.

Bibliography

- [1] George E. P. Box, Gwilym M. Jenkins, Gregory C. Reinsel, and Greta M. Ljung. *Time Series Analysis: Forecasting and Control*. John Wiley & Sons, Hoboken, NJ, 5 edition, 2015.
- [2] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [3] Eduardo F. Camacho and Carlos Bordons. *Model Predictive Control*. Springer, London, 2 edition, 2007.
- [4] A. Charnes and W. W. Cooper. Chance-constrained programming. *Management Science*, 6(1):73–79, 1959.
- [5] Christina Delimitrou and Christos Kozyrakis. Paragon: QoS-aware scheduling for heterogeneous datacenters. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '13)*, pages 77–88. ACM, 2013.
- [6] Panagiotis Garefalakis, Konstantinos Karanasos, Peter Pietzuch, Arun Suresh, and Sriram Rao. Medea: Scheduling of long running applications in shared production clusters. In *Proceedings of the Thirteenth EuroSys Conference (EuroSys '18)*. ACM, 2018.
- [7] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [8] Kubernetes. Kubernetes documentation. <https://kubernetes.io/docs/home/>, 2026. Accessed: 2026-05-02.
- [9] D. Q. Mayne, M. M. Seron, and S. V. Raković. Robust model predictive control of constrained linear systems with bounded disturbances. *Automatica*, 41(2):219–224, 2005.
- [10] James B. Rawlings, David Q. Mayne, and Moritz M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, Santa Barbara, CA, 2 edition, 2024. Fifth printing.
- [11] Krzysztof Rządca, Paweł Findeisen, Jacek Swiderski, Przemysław Zych, Przemysław Broniek, Jarek Kusmieriek, Paweł Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. Autopilot: Workload autoscaling at Google. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys '20)*, pages 16:1–16:16. ACM, 2020.

- [12] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.
- [13] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.
- [14] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15)*, pages 18:1–18:17. ACM, 2015.
- [15] Luping Wang, Qizhen Weng, Wei Wang, Chen Chen, and Bo Li. Metis: Learning to schedule long-running applications in shared container clusters at scale. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–17. IEEE, 2020.
- [16] Timothy Wood, Prashant Shenoy, Arun Venkataramani, and Mazin Yousif. Sandpiper: Black-box and gray-box resource management for virtual machines. *Computer Networks*, 53(17):2923–2938, 2009.
- [17] Fei Xu, Xiyue Shen, Shuohao Lin, Li Chen, Zhi Zhou, Fen Xiao, and Fangming Liu. Tetris: Proactive container scheduling for long-term load balancing in shared clusters. *IEEE Transactions on Services Computing*, 17(5):2918–2930, 2024.
- [18] Ding Zou, Wei Lu, Zhibo Zhu, Xingyu Lu, Jun Zhou, Xiaojin Wang, Kangyu Liu, Kefan Wang, Renen Sun, and Haiqing Wang. OptScaler: A collaborative framework for robust autoscaling in the cloud. *Proceedings of the VLDB Endowment*, 17(12):4090–4103, 2024.